CNN/Siamese Embedding Endpoints

Integration guide for the Flutter team — a second, independent single-finger matching path alongside the existing minutiae endpoints

1. What this is

Three new endpoints on the same cloud service you already talk to, offering an alternative to minutiae-based matching: a CNN (pretrained ResNet18) converts each fingerprint image into a 512-number vector ("embedding"), and two images are compared by how similar their vectors are (cosine similarity), instead of comparing extracted ridge-point locations. Validated at 92.5% matching accuracy on a 336-person test set (vs. ~52% for the existing minutiae matcher) — see the attached *Siamese_Embedding_Phase_A_Report.pdf* for the full methodology and numbers.

This runs **alongside** the existing endpoints, not instead of them. Nothing about /v2/enroll, /v2/authenticate, /v2/verify changes — they still work exactly as before, on their own separate data table.

2. Base URL

`https://34-100-150-103.sslip.io`

Same host as the existing /v2/* minutiae endpoints.

3. Endpoints

POST /v2/enroll_embedding

Multipart form fields:

Field	Required	Notes
image	Yes	The preprocessed fingerprint image (same as you'd send to /v2/enroll)
name	Yes	Person's name
uid	Yes	Unique person identifier
batch	Yes	Batch/group identifier — authenticate matches within one batch
finger_position	No	Integer 0-10, default 0

Response (success):

```
{"success": true, "embedding_dim": 512}
```

Response (error, e.g. missing field or bad image):

```
{"success": false, "error": "..."}
```

POST /v2/authenticate_embedding — 1:N (identify)

Multipart fields: **batch**, **image**. Compares the image against every person enrolled in that batch, returns the best match.

Response (match found):

```
{"success": true, "name": "...", "uid": "...", "confidence": 0.9954, "threshold": 0.94}
```

Response (no match / below threshold):

```
{"success": false, "message": "No match found", "confidence": 0.6123, "threshold": 0.94}
```

POST /v2/verify_embedding — 1:1 (confirm identity)

Multipart fields: **uid**, **batch**, **image**. Compares the image only against that specific person's enrolled embedding.

Response:

```
{"success": true, "matched": true, "confidence": 0.9954, "threshold": 0.94, "name": "...",
"uid": "..."}
```

Response (uid not found in batch):

```
{"success": false, "error": "No user found: <uid>"} — HTTP 404
```

4. Reading the response

confidence is a cosine similarity score from -1 to 1 (in practice you'll see roughly 0.85-1.0 for fingerprint photos, genuine or not — this embedding space sits in a narrow, high band, not spread across the full range). **matched** / whether a candidate is returned at all is decided by comparing confidence against **threshold** (currently 0.94, also returned in every response so the app never needs to hardcode it).

5. Important — threshold is not fully calibrated yet

0.94 was measured on a lab dataset (HKPU contactless fingerprint database — different camera, lighting, and preprocessing than this app's actual capture pipeline). It's a reasonable starting point, not a final production number. Expect this value to change once it's re-measured against real captures from the app — build against the **threshold** field in the response rather than hardcoding 0.94 anywhere in the app, so a server-side recalibration doesn't require a client update.

6. Example (curl)

Enroll:

```
curl -X POST https://34-100-150-103.sslip.io/v2/enroll_embedding \
  -F "name=Jane Doe" -F "uid=U001" -F "batch=demo" \
  -F "image=@fingerprint.jpg"
```

Verify:

```
curl -X POST https://34-100-150-103.sslip.io/v2/verify_embedding \
  -F "uid=U001" -F "batch=demo" -F "image=@fingerprint2.jpg"
```

7. Which one should the app actually call?

Both matchers are live and independent — the minutiae ones and these embedding ones don't share data, so enrolling in one doesn't enroll in the other. If you want to compare them side-by-side in the app during evaluation, call both endpoints with the same image and show both results; for a single production flow, use whichever one the team decides on after the threshold is recalibrated on real data (see §5) — that decision isn't made yet, so don't hardcode a permanent choice.

Backend: backend/embedding_matcher.py + backend/cloud_app.py (branch siamese-embedding). Validation report: siamese_embedding/Siamese_Embedding_Phase_A_Report.pdf.